

Graduation Project Specifications & SRS

Guidelines

Document Version:	1.0 (Official Specification)
Target Audience:	Full-Stack Python Trainees & Project Advisors
Final Deadline:	29 September 2026 (Offline / In-Person Evaluation)
Core Stack:	Python 3.12, Django (MVT), PostgreSQL, JS/ES6+, Agentic AI

taking real action within application bounds. This project explicitly introduces Agentic AI concepts into your tech stack.

- t Design  **CORE AI PARADIGM REQUIREMENT**
CRITICAL AI REQUIREMENT: Your graduation project must NOT feature a isolated chatbot that simply answers general knowledge questions or functions as an isolated text box.mponent**1.**

Project Context & Overview

Welcome to the capstone phase of the Summer Training Full-Stack Python Program. Over the past weeks, you have built foundational and advanced skills across the complete web development lifecycle. The graduation project is the culmination of your journey—a comprehensive software engineering challenge designed to transform theoretical concepts into a production-grade, commercial-quality software product.

Your technical mastery encompasses the following core stack:

- Frontend Engineering: HTML5, CSS3, Modern JavaScript (ES6+), Dynamic DOM Manipulation.
- Backend Architecture: Python 3.12+, Django Framework utilizing the MVT (Model-View-Template) pattern.
- Database Systems: PostgreSQL, Relational Schema Design, Data Integrity, Constraints, and Django ORM optimization.
- AI & Prompt Engineering: Structured Promp, System Instructions, Non-RAG Context Management, Vibe Coding productivity.

1.1 The Paradigm Shift: Introducing Agentic AI

- Modern web software is evolving rapidly beyond static databases and simple text-in/text-out chat interfaces. Today's applications incorporate autonomous, goal-oriented AI cos capable

Teams are strictly required to implement an Agentic AI Workflow. The AI component must be given a high-level user goal, reason about the task, select appropriate backend tools/actions, interact directly with your application's database and services, and return validated, structured outcomes.

Note on Infrastructure: To keep focus on software engineering, clean code, and tool execution, the project does NOT require Retrieval-Augmented Generation (RAG), vector databases, embeddings, or complex cloud orchestration frameworks. The agentic flow must be achieved via standard LLM Tool Calling (Function Calling) integrated with custom Django business logic.

2. Primary Project Goals & Learning Objectives

The objective is to deliver a complete, fully integrated web application that addresses a real-world domain problem. Trainees must treat this assignment as a professional software product engineering cycle.

- **Full-Stack Integration:** Seamlessly link PostgreSQL, Django ORM, server-rendered views/templates, and interactive frontend ES6+ modules.
- **Robust Authentication & Permissions:** Enforce strict security models separating user roles and securing backend endpoints.
- **Agentic AI Execution:** Design safe, deterministic tool integrations where an AI agent executes CRUD tasks on behalf of authenticated users.
- **Professional Engineering Workflow:** Maintain continuous Git histories, structured GitHub repositories, pull requests, issue tracking, and strict code review habits.
- **Production Readiness:** Deliver clean documentation, comprehensive Software Requirements Specifications (SRS), optimized database schemas, and defect-free execution.

3. Core Technical & Functional Requirements

3.1 Web Application Architecture

The project must be built using Python, Django (MVT pattern), PostgreSQL, HTML5, CSS3, and modern ES6+ JavaScript. Django templates should serve clean HTML structured with semantic layout standards. Frontend interactivity (dynamic UI updates, asynchronous fetch calls to endpoints, modal handles, realtime form feedback) must be handled clean JavaScript modules.

3.2 Authentication, Authorization & Role-Based Access Control (RBAC)

Security is mandatory. Applications must handle user identity lifecycle robustly:

- Registration & User Lifecycle: User registration with password hashing, login, logout, and profile management.
- Role-Based Access Control (RBAC): System must support multi-tenancy or distinct roles (e.g., Administrator, Manager, Standard User, Client).
- Granular Endpoint Security: Django `@login_required`, `UserPassesTestMixin`, or explicit view permission checks must prevent horizontal or vertical privilege escalation.
- Agent Context Security: The AI agent must inherit the logged-in user's credentials and permissions. An agent acting for a Standard User must never execute Admin-level actions.

3.3 Database Design & Data Integrity (PostgreSQL)

PostgreSQL serves as the single source of truth. Monolithic or flat tables are strictly prohibited. Teams must model a normalized relational database (minimum 3NF recommended where appropriate) demonstrating:

- Well-defined Entities with appropriate primary keys, foreign keys, unique constraints, and indexes.
- Clear Cardinality: One-to-Many (1:N) and Many-to-Many (M:N) relationships utilizing Django `ManyToManyField` through tables when attributes exist.
- Data Integrity Rules: DB-level field constraints (`CheckConstraint`, default values, nullability rules) preventing corrupt state creation.
- Documented Schema Rationale: The SRS must explicitly explain why tables were normalized and how business rules dictated schema decisions.

4. Context-Aware AI Chatbot Specification

Every project must feature a conversational UI module integrated into the web dashboard. Unlike standalone AI interfaces, this chatbot is embedded within the Django runtime context.

Key Chatbot Responsibilities:

- Context Awareness: The system dynamically feeds active application state (e.g., user profile, active tasks, cart contents, order status) into the conversation context.
- Guided User Support: Assisting users in navigating complex app features, providing domain-specific answers grounded in application rules.
- Content Generation & Summarization: Summarizing user report logs, drafting communications, or analyzing dashboard metrics.
- Error & Query Handling: Gracefully managing invalid inputs and guiding the user toward valid system interactions.

5. Agentic AI & Tool-Calling Architecture

This requirement represents the defining technical milestone of your graduation project. Each team must implement an Agentic Workflow capable of autonomous reasoning, tool selection, and application side-effect execution.

5.1 Conceptual Agentic Loop

The agent must operate according to the following control loop:

- 1. Goal Perception:** User submits a high-level intent (e.g., 'Reschedule my pending low-priority tasks to next Monday and notify the lead').
- 2. Reasoning & Tool Selection:** The LLM inspects available Python/Django tool definitions (JSON schemas) and selects the matching action(s).
- 3. Action Execution:** The Django backend receives the structured function call, verifies permissions, and executes the backend Python function.
- 4. Observation & Feedback:** The backend returns the execution output (success/failure data) back to the LLM.
- 5. Task Completion:** The LLM synthesizes the final response and presents the outcome clear to the user.

6. AI Safety, Tool Governance & Permissions

Giving an AI agent execution rights introduces severe security risks if unmonitored. AI Agents **MUST NEVER** have raw SQL access or unvalidated database privileges.

AI GOVERNANCE RULES

SECURITY MANDATE: Mandatory Validation Framework

1. Identity Binding: Every tool call must pass the `request.user` session context to the tool function.
2. Permission Check: Tool functions must explicitly run permission verifications (e.g., checking ownership or `user.has\_perm`).
3. Parameter Validation: Tool inputs must be validated against typing rules and standard Django Form / Pydantic schemas before execution.
4. Destructive Action Confirmation: Critical operations (e.g., permanent deletion, financial transaction) **MUST** require explicit user UI confirmation or a two-step validation flag.

7. Prompt Engineering & System Design

Prompt engineering is formal software design. Copying raw prompts or utilizing generic system instructions is strictly unacceptable. Trainees must document their Prompt Engineering process in the SRS.

Prompt Documentation Requirements in SRS:

- System Persona & Guardrails: Explicit instructions specifying identity, tone, forbidden topics, and refusal boundaries.
- Tool Definitions: Precise documentation of tool descriptions provided to the LLM to prevent hallucinated argument passing.
- Prompt Evolution Log: Demonstration of how prompts were iteratively refined from initial failures to robust execution.

- Output Formatting Controls: Enforcing structured JSON responses or specific Markdown formats.

8. Vibe Coding Guidelines & Accountability

Vibe Coding—the modern practice of leveraging AI coding assistants (e.g., Claude, Cursor, ChatGPT, Copilot) to rapidly generate boilerplate and prototype features—is permitted and encouraged as a developer productivity booster.

 **VIBE CODING PROTOCOL**

DEVELOPER RESPONSIBILITY RULE:

While AI may assist in generating code, human trainees bear 100% legal, technical, and academic responsibility for every line of code committed to the repository.

During evaluation, instructors WILL conduct code defense examinations. You must be able to explain the logic, refactor code on the spot, debug errors, and justify architectural decisions. Blindly accepting AI generated code without full comprehension will result in immediate scoring penalties.

9. Git & GitHub Professional Workflow

Software engineering is a collaborative craft. Your GitHub commit graph and development workflow constitute a major portion of your final grade. Repository creation must happen on Day 1.

9.1 Anti-Pattern Warning: The Bulk Upload

Uploading a complete codebase 2 hours before the deadline is strictly forbidden and will be treated as a critical workflow failure, incurring up to a 10-point penalty in the evaluation rubric.

9.2 Repository Expectations & Commit Conventions

Teams must follow Conventional Commit Standards:

Commit Message Pattern	Description
feat: add user auth & profile schema	Standard feature addition commit
fix: validate agent tool input parameters	Bug fix commit with detailed context
feat: connect task creation agent tool	Integration commit for AI tools
docs: update setup steps and ERD details	Documentation update commit

10. Recommended Django Project Structure

Maintain clean separation of concerns across Django applications. Avoid placing all logic in a single monolith application module.

```
my_project/
├── manage.py
├── requirements.txt
├── README.md
├── .env.example
├── config/                # Project Root Configuration
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── apps/
│   ├── authentication/   # Custom User, RBAC, Profiles
│   ├── core/             # Base Layouts, Landing Pages
│   ├── domain_app/       # Core Business Models, Views, Forms
│   └── ai_agent/          # Chatbot Interface, Agent Logic, Tool Registry
│       ├── services.py    # LLM & Agent loop logic
│       ├── tools.py        # Python Executable Tools for Agent
│       └── prompts.py     # Centralized System Prompts
├── static/
│   ├── css/
│   └── js/
│       └── agent_chat.js  # Async Fetch API Chat UI Module
```

11. Mandatory SRS Document Structure

Each team must submit a standalone Software Requirements Specification document (`SRS.pdf` or `SRS.docx` inside `/docs`). The SRS must include the following sections:

- **1. Introduction:** Title, detailed Problem Statement, Project Objective, Target User Personas, System Scope.
- **2. Functional Requirements:** Formal requirements using standard notation ('The system shall allow users to...').
- **3. Non-Functional Requirements:** Performance, Security, Usability, Reliability, Data Consistency benchmarks.
- **4. User Roles & RBAC Matrix:** Grid defining permissions per user role across all module endpoints.
- **5. System Architecture Diagram:** Visual schematic depicting Frontend <-> Django <-> PostgreSQL <-> AI Agent Service.
- **6. Database Design & ERD:** Entity-Relationship Diagram with attribute lists, PK/FK definitions, and 3NF normalization notes.
- **7. User Journey Flows:** Step-by-step visual or narrative maps of key task executions.
- **8. AI & Agent Design:** Detailed tool descriptions, argument schemas, security boundaries, and sample agent traces.
- **9. GitHub & Development Strategy:** Branching model, commit rules, and issue board organization.

12. Deliverables & Demonstration Requirements

At the final evaluation session on 22 September 2026, teams must provide:

- Fully Functional Web Application running in local production mode with seed data loaded.
- PostgreSQL Database dump/migrations script populated with realistic scenario data.
- Public GitHub Repository featuring continuous commit logs, active PRs, and closed issues.
- Complete SRS Specification Document & Rendered ERD / Architecture Diagrams.
- Professional README.md containing complete local environment setup steps (`pip install`, `migrate`, `.env` key guide).
- Live 15-Minute Project Defense & Technical Demonstration.

13. Comprehensive Evaluation Rubric (100 Points)

Projects will be strictly evaluated by instructors based on the following official 100-Point Rubric:

Evaluation Category	Pts	Evaluation Criteria & Focus Areas
1. Problem & Idea	10	Problem clarity, target user relevance, practical utility, realistic project scope.
2. Frontend UI/UX	10	HTML5 structure, clean CSS styling, responsive layout, ES6+ JS interaction & async fetch.
3. Django & MVT	15	Proper MVT architecture, clean views/templates/forms, separation of business logic.
4. Database & Postgres	10	Schema normalization, ERD alignment, ORM query efficiency, foreign keys & indexes.
5. Auth & Security	5	Authentication flows, role permissions (RBAC), secure tool validation, CSRF protection.
6. AI Chatbot UI	10	Context integration, clean conversational interface, useful output, robust prompts.
7. Agentic AI & Tools	15	Real tool execution, multi-step reasoning, controlled actions, safety & validation checks.
8. Git & GitHub Flow	10	Continuous commit history, branch usage, PRs, task/issue tracking, quality README.
9. SRS & Docs	5	Completeness of SRS, accurate ERD & architecture diagrams, installation clarity.
10. Presentation & Defense	10	Live demo quality, individual technical understanding, code modification defense capability.

14. Teamwork Guidelines & Timeline

14.1 Team Structure & Responsibilities

Teams should ideally consist of 3–4 trainees. While roles may be assigned (Frontend Lead, Django Backend Lead, DB/Agent Engineer), all team members must possess full comprehension of the entire codebase.

16. Exemplar Project Ideas & Agentic Use Cases

Teams may select one of the following exemplar project ideas or propose their own equivalent product idea subject to instructor approval.

1. Smart Car Service & Maintenance System

A comprehensive platform for managing vehicles, maintenance records, service appointments, technicians, and spare parts.

Agentic AI Use Case:

The AI Agent reviews the user's vehicle maintenance history, identifies required or overdue services, checks available service slots, and can book a suitable maintenance appointment through validated backend tools.

2. Smart Delivery & Logistics Management System

A logistics platform for managing deliveries, customers, drivers, delivery assignments, and shipment statuses.

Agentic AI Use Case:

The AI Agent analyzes delayed or pending deliveries, checks available drivers, and can reassign deliveries or update shipment statuses through permission-checked backend tools.

3. Smart Facility & Maintenance Management System

A facility management platform for handling buildings, maintenance requests, reported issues, technicians, spare parts, and maintenance schedules.

Agentic AI Use Case:

The AI Agent identifies critical unresolved maintenance requests, checks available technicians, and can assign maintenance tasks or update their status through validated Django tools.

Important:

Each project must implement the same core technical requirements defined in this specification, including Django MVT, PostgreSQL, authentication and RBAC, context-aware AI chatbot, Agentic AI with Tool Calling, parameter validation, permission checks, Git/GitHub workflow, and complete SRS documentation.

17. Final Submission Verification Checklist

Before final submission on 22 September 2026, teams must verify that all elements are complete:

- ☐ Working Django Web Application (built on Python 3.12+ & Django MVT)
-

- ☐ PostgreSQL Database (fully schema-normalized with migrations)
- ☐ Authentication & Role-Based Access Control (RBAC) active
- ☐ HTML5 / CSS3 / ES6+ JavaScript frontend implementation
- ☐ Context-Aware AI Chatbot UI embedded in application
- ☐ Agentic AI Workflow implemented with dynamic Tool-Calling
- ☐ At least 1-2 fully functional backend tool functions executed by Agent
- ☐ Strict input parameter validation & user permission checks on AI tools
- ☐ Complete Software Requirements Specification (SRS) document
- ☐ Rendered Entity-Relationship Diagram (ERD) & Architecture Diagram
- ☐ GitHub Repository created from project kickoff with continuous commits
- ☐ Repository structured with Branches, Pull Requests, and Issue Tracking
- ☐ Professional README.md with comprehensive installation instructions
- ☐ All team members prepared for individual code defense & live demo

18. Poll Options:

(AI Agent aggregates transaction history, flags over-budget spending, and sets automated financial alerts).

💡 Note to Include:

⚠️ **Important:** Choose a domain you clearly understand and feel confident implementing safe, permission-checked AI agent tool executions for.

Group 1 1. esraa shalaby 2. mena ibrahim hassan	Group 2 1. habiba sameer 2. fatma mohamd	Group 3 1. nada wesam 2. noor mohamed	Group 4 1. abdelhamid ajmed 2. ahmed adel
Group 5 1. maryam mohamed 2. amr allam	Group 6 1. arwa mahmoud 2. abdelrahman atef	Group 7 1. khaled sameh 2. omar samir	Group 8 1. khaled abdelmajid 2. menna khattab
Group 9 1. malak osama 2. Abdallah wael	Group 10 1. mai kamal 2. onia ayman	Group 11 1. ghada abdelbaki 2. mostafa sherif	

--- END OF OFFICIAL SPECIFICATION ---